

# Flask

Websites & Model Deployment

# Why Flask?

- Flask is microframework (minimal core) with three main components: **a) routing** **b) templates** **c) requests**
- One add libraries (ORM, auth, forms) on need. Great for small services, microservices, **quick ML model serving**.
- **Routing:** `@app.route('/path')` decorators on functions to attach an address to it. This address can be found via URLs. Functions can be regarded as views/controllers.
- **Templates:** (Jinja): Frontend files to serve user when a particular url is called.
- **Requests:** Secured communication between templates and views.

# Deploying ML/DL model with Flask

# Virtual Environment

- Create a virtual environment first:

```
python -m venv mvenv
```

```
# Windows
```

```
mvenv\Scripts\activate
```

```
# mac/linux
```

```
# source venv/bin/activate
```

# Project Structure

ml-flask/           → (Folder name)

- |— model/       → (models .h5 or .bib files)
- |— app.py       → (Flask main file containing routes and views)
- |— requirements.txt → (libraries and their versions to install)
- |— templates/   → (html files, each html file will represent a view)
  - |   |— base.html
  - |   └— index.html
- └— static/       → (CSS, JS files are regarded as static files)
  - |— style.css
  - └— predict.js

# Requirements.txt

numpy

pandas

scikit-learn

joblib

matplotlib

flask

gunicorn

tensorflow

pillow

To install all the packages at once:

```
pip install -r requirements.txt
```

# Create Hello World App

**Step 1:** Create **app.py** file and Initialize flask instance:

```
from flask import Flask
```

```
# Create a Flask application instance
```

```
app = Flask(__name__)
```

To run the project:

```
python app.py
```

```
# Main function. Should be the last statement of file
```

```
if __name__ == "__main__":
```

```
    # For local development
```

```
    app.run(host="127.0.0.1", port=5000, debug=True)
```

# Create Hello World App

## Step 2: Create first route

```
@app.route('/')          # use '/' to create a default route
def hello_world():
    return "<h1>Hello, World! Welcome to Flask.</h1>"
```

To run the project:

```
python app.py
```



# Create Hello World App

## Step 3: Create second route

```
@app.route('/')          # use '/' to create a default route
def hello_world():
    return "<h1>Hello, World! Welcome to Flask.</h1>"
```

```
@app.route("/msg")      # msg route to call message view/controller.
def message():
    return "<h1>Say, Hello again! </h1>"
```

To run the project:

```
python app.py
```

# Create Hello World App

## **Step 4:** Render an html template

```
from flask import Flask, render_template
```

```
@app.route("/")
```

```
def home():
```

```
    return render_template("index.html") → templates/index.html
```

To run the project:

```
python app.py
```

# Create Hello World App

**Step 5:** Sending data from flask to html

**controller**

```
@app.route("/user")
def user():
    name = "Usman"
    return render_template("user.html", username=name)
```

To run the project:

```
python app.py
```

# Create Hello World App

**Step 5:** Sending data from flask to html

index.html

```
<body>  
  <h1>Hello {{ username }}!</h1>  
</body>
```

To run the project:

```
python app.py
```

# Create Hello World App

**Step 5:** Sending data from flask to html

index.html

```
<body>  
  <h1>Hello {{ username }}!</h1>  
</body>
```

To run the project:

```
python app.py
```

# HTTP Methods and their meaning:

| Method       | Meaning                            |
|--------------|------------------------------------|
| GET          | Read data                          |
| POST         | Insert data                        |
| PUT or PATCH | Update data, or insert if a new id |
| DELETE       | Delete data                        |

## GET

- Values are contained in the URL;
- Has a length limitation of 255 characters;
- It is often cacheable;
- Supports only string data types;
- Parameters are saved in browser history;
- Results can be bookmarked.

## POST

- Values are contained in the message's body;
- Does not have a length limitation ;
- It is hardly cacheable;
- Supports different data types, such as string, numeric, binary, etc;
- Parameters are not saved in browser history;
- Results cannot be bookmarked.

# Create Hello World App

## Step 5: Handling Post Request

```
@app.route("/submit", methods=["POST"])  
def submit():  
    user_name = request.form["username"]  
    return render_template("result.html", name=user_name)
```

To run the project:

```
python app.py
```

# Create Hello World App

## Step 6: Handling Post Request

**controller**

```
@app.route("/submit", methods=["POST"])
def submit():
    user_name = request.form["username"]
    return render_template("result.html", name=user_name)
```

To run the project:

```
python app.py
```



# Create Hello World App

## **Step 6:** Handling Post Request

**index.html**

```
from flask import request
```

```
@app.route("/submit", methods=["POST"])
```

```
def submit():
```

```
    user_name = request.form["username"]
```

```
    return render_template("result.html", name=user_name)
```